



Python基本数据类型

- 整数类型
- 浮点数类型
- 复数类型
- 数值运算操作符
- 运算符优先级
- 增强赋值操作符
- 数字类型的转换
- 内置的数值运算函数
- Python内置函数、标准库、第三方库
- math 库 & random库

数字类型

- 概念：表示数字或数值的数据类型
- 计算机对数字的识别和处理的基本要求：
 - ✓ **确定性**：正确无歧义地解读数据所代表的类型含义

1010：整数值1010？门牌号一样的字符串？
二进制？十进制？
 - ✓ **高效性**：程序为数字运算提供较高的计算速度，较少的存储空间代价

整数加法和小数加法，前者比后者运算速度快5-20倍

■ Python语言包括三种数字类型

- 整数类型
- 浮点数类型
- 复数类型

与数学中的整数概念一致，有4种进制表示

- **十进制**：1010, 99, -217

- **二进制**：每一位只有 0 和 1 两个取值。Python 中以 **0b** 或 **0B** 为前缀，例如，0b010, -0B101

- **八进制**：每一位的取值为 0–7，共 8 个数字。Python 中以 **0o** 或 **0O** 为前缀，例如，0o123, -0O456

- **十六进制**：每一位的取值为 0–9 和 A–F（或 a–f），共 16 个符号，其中 A=10, B=11, ..., F=15。Python 中以 **0x** 或 **0X** 为前缀，例如，0x9a, -0X89

■ 十进制 → 其他进制（除基取余法）

- **核心思路**：不断除以目标进制的基数，记录余数，最后将余数**从下往上**排列
- 十进制 26 转二进制：

步骤	除法	商	余数
1	$26 \div 2$	13	0
2	$13 \div 2$	6	1
3	$6 \div 2$	3	0
4	$3 \div 2$	1	1
5	$1 \div 2$	0	1

余数从下往上读：**11010**，即 $26 = 0b11010$

■ 十进制 → 其他进制（除基取余法）

- **核心思路**：不断除以目标进制的基数，记录余数，最后将余数从下往上排列
- 十进制 26 转十六进制：

步骤	除法	商	余数
1	$26 \div 16$	1	10（即 A）
2	$1 \div 16$	0	1

余数从下往上读：**1A**，即 $26 = 0x1A$

■ 其他进制 → 十进制（按权展开法）

- **核心思路**：权重 = 基数的幂次，幂次由该位的位置决定——**从右往左，从 0 开始递增**。将每一位的数字 × 对应权重，最后求和
- 二进制 → 十进制（基数为 2，以 0b11010 为例）：

位置（从右往左）	第 4 位	第 3 位	第 2 位	第 1 位	第 0 位
数字	1	1	0	1	0
权重 ($2^{\text{位置}}$)	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
数字 × 权重	16	8	0	2	0

$$1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 16 + 8 + 0 + 2 + 0 = 26$$

■ 其他进制 → 十进制（按权展开法）

- **核心思路**：权重 = 基数的幂次，幂次由该位的位置决定——**从右往左，从 0 开始递增**。将每一位的数字 × 对应权重，最后求和
- 八进制 → 十进制（基数为 8，以 0o32 为例）：

位置（从右往左）	第 1 位	第 0 位
数字	3	2
权重 ($8^{\text{位置}}$)	$8^1 = 8$	$8^0 = 1$
数字 × 权重	24	2

$$3 \times 8^1 + 2 \times 8^0 = 24 + 2 = 26$$

■ Python 内置进制转换 (十进制 → 其他进制)

bin(): built-in function, 将整数转换为二进制字符串, 前缀为 '0b'

```
bin(26)      # '0b11010'
```

oct(): built-in function, 将整数转换为八进制字符串, 前缀为 '0o'

```
oct(26)      # '0o32'
```

hex(): built-in function, 将整数转换为十六进制字符串, 前缀为 '0x'

```
hex(26)      # '0x1a'
```

如果不想要前缀, 可以用切片去掉前两个字符

```
bin(26)[2:]   # '11010'
```

```
oct(26)[2:]   # '32'
```

```
hex(26)[2:]   # '1a'
```

■ Python 内置进制转换 (其他进制→十进制)

`int(string, base)`: 将字符串按指定的进制 `base` 解析为十进制整数

参数1: 要转换的数字字符串

参数2: 该字符串所表示的进制 (2, 8, 16 等, 范围 2~36)

`int('11010', 2)` # 26 — 把 '11010' 当作二进制, 转为十进制

`int('32', 8)` # 26 — 把 '32' 当作八进制, 转为十进制

`int('1A', 16)` # 26 — 把 '1A' 当作十六进制, 转为十进制

■ Python 内置进制转换 (其他进制→十进制)

- Python支持在源代码中直接书写各进制的字面量 (literal)
- 无论用哪种写法，在内存中存储的都是同一个整数

```
x = 0b11010    # 二进制字面量, x = 26
y = 0o32        # 八进制字面量, y = 26
z = 0x1A        # 十六进制字面量, z = 26
```

```
# 验证：它们完全相等
```

```
print(x == y == z)    # True
print(type(x))         # <class 'int'>
```

整数类型取值范围

⚠ 理论上取值范围没有限制，实际受限于内存大小

➤ 利用 `pow(x, y)` 函数测试：

两参数形式：`pow(base, exp)`

计算 `base` 的 `exp` 次方，等价于 `base ** exp`。

当 `base` 和 `exp` 都是整数时，返回 `int`；当 `exp` 为负数时，返回 `float`。

示例	含义	结果
<code>pow(2, 10)</code>	2^{10}	1024
<code>pow(3, 4)</code>	3^4	81
<code>pow(5, 0)</code>	5^0	1（任何数的 0 次方等于 1）
<code>pow(2, -1)</code>	2^{-1}	0.5（负指数返回 float）

等价写法：`2 ** 10` 与 `pow(2, 10)` 功能完全相同，`**` 是 Python 的幂运算符。

整数类型取值范围

⚠ 理论上取值范围没有限制，实际受限于内存大小

➤ 利用 `pow(x, y)` 函数测试：

`pow(2, 10)` # 1024 (千级别)

`pow(2, 15)` # 32768 (万级别)

`pow(2, 1000)` # 一个 302 位的整数

`pow(2, pow(2, 15))` # 即 2^{32768} ，一个 9865 位的整数

➤ 利用 `sys.getsizeof()` 展示内存开销：

- 是 Python 标准库 `sys` 模块中的函数
- 用于返回一个对象在内存中占用的字节数
- 使用前需要先导入：`import sys`

整数类型取值范围

⚠ 理论上取值范围没有限制，实际受限于内存大小

➤ 利用 `pow(x, y)` 函数测试：

`pow(2, 10)` # 1024 (千级别)

`pow(2, 15)` # 32768 (万级别)

`pow(2, 1000)` # 一个 302 位的整数

`pow(2, pow(2, 15))` # 即 2^{32768} ，一个 9865 位的整数

➤ 利用 `sys.getsizeof()` 展示内存开销：

<code>sys.getsizeof(0)</code>	28 字节	整数 0 的基础开销
<code>sys.getsizeof(1)</code>	28 字节	小整数，和 0 一样
<code>sys.getsizeof(2**10)</code>	28 字节	1024，仍是小整数
<code>sys.getsizeof(2**1000)</code>	164 字节	302 位的大整数，明显变大
<code>sys.getsizeof(2**pow(2,15))</code>	约 14516 字节	9865 位的巨大整数，约 14 KB

整数类型取值范围

⚠ 理论上取值范围没有限制，实际受限于内存大小

➤ 与 C/Java 对比

语言	整数类型	位宽	取值范围
C	<code>int</code>	通常 32 位	$-2^{31} \sim 2^{31} - 1$ (约 ± 21 亿)
Java	<code>int</code>	32 位	$-2^{31} \sim 2^{31} - 1$
Java	<code>long</code>	64 位	$-2^{63} \sim 2^{63} - 1$
Python 3	<code>int</code>	无固定位宽	仅受内存限制

在 C/Java 中超过范围会发生**溢出**，而 Python 不会

浮点数类型

- 与数学中实数概念一致，表示带有小数的数值

⚠ 浮点数必须带有小数部分（可以是0）

示例：0.0，-77.，-2.17

- 表示方法：十进制和科学计数法

➤ 十进制：直接写带小数点的数

➤ 科学计数法以10为基数，使用字母“e”或者“E”作为幂的符号。

$$\langle a \rangle e \langle b \rangle = a * 10^b$$

示例：0.0043 = 4.3e-3

浮点数类型

■ 存储原理

- 计算机内部把每个浮点数都表示成二进制科学计数法： $1.xxxxx \times 2^n$
- 用 64 个二进制位（8 字节）存储一个浮点数
- 64 位的分配：

段	位数	存什么
符号位	1 位	正数还是负数（0 = 正，1 = 负）
指数	11 位	二进制科学计数法中的 n ，决定数有多大/多小
尾数	52 位	二进制科学计数法中小数点后的部分，决定具体是哪个数

⚠ 存储空间有限，所以浮点数有**范围限制**，也有**精度限制**。
这是和 Python 整数最大的区别



浮点数类型

■ 三个核心限制

```
import sys
print(sys.float_info.max)           # 1.7976931348623157e+308
print(sys.float_info.min)           # 2.2250738585072014e-308
print(sys.float_info.epsilon)       # 2.220446049250313e-16
```

限制	值	通俗理解
最大值 <code>max</code>	$\approx 1.8 \times 10^{308}$	超过这个值会变成 <code>inf</code> (无穷大)
最小正值 <code>min</code>	$\approx 2.2 \times 10^{-308}$	比这更接近 0 的正数会"下溢"为 0
精度 <code>epsilon</code>	$\approx 2.2 \times 10^{-16}$	在 1.0 附近, 比这更小的变化会被忽略

说明在 1.0 这个位置, 小于 $\text{epsilon} / 2 \approx 1.11 \times 10^{-16}$ 的变化量, 计算机感知不到, 加了等于没加

浮点数类型

■ 精度 epsilon

在 1.0 附近，数轴上相邻的两个浮点数是：

1.0	(当前刻度)
1.00000000000000002	(下一个刻度, 间距 = epsilon)

中间没有任何其他浮点数可以表示。所以当你算 $1.0 + \text{一个小数}$ 时，结果只能二选一：要么落到 1.0 ，要么落到 1.00000000000000002 。

怎么选？看中点

就像四舍五入一样，计算机看你离哪个刻度更近：

1.0	中点	1.00000000000000002
----- -----		
← 离1.0近, 舍		→ 离下一个近, 入

中点 = $1.0 + \text{epsilon} / 2 \approx 1.0 + 1.11 \times 10^{-16}$



浮点数类型

■ 精度 epsilon

<code>print(1.0 + 1.0e-16)</code>	<code># 1.0</code>	← 被忽略, 离 1.0 更近
<code>print(1.0 + 1.1e-16)</code>	<code># 1.0</code>	← 被忽略, 离 1.0 更近
<code>print(1.0 + 1.11e-16)</code>	<code># 1.0</code>	← 被忽略, 几乎在中点, 舍入到 1.0
<code>print(1.0 + 1.12e-16)</code>	<code># 1.0000000000000000002</code>	← 加得上! 刚超过中点
<code>print(1.0 + 1.2e-16)</code>	<code># 1.0000000000000000002</code>	← 加得上
<code>print(1.0 + 2.2e-16)</code>	<code># 1.0000000000000000002</code>	← 加得上
<code>print(1.0 + epsilon)</code>	<code># 1.0000000000000000002</code>	← 加得上, 刚好到下一个刻度

■ 精度限制

- 可以精确表示 **15** 位有效数字
- **有效数字**：从左边第一个不是 0 的数字起，到精确的位数止，所有的数字

15 位以内：准确

```
x = 0.123456789012345
```

```
print(x)    # 0.123456789012345  ✓
```

第 16 位起：不可信

```
y = 0.1234567890123456789
```

```
print(y)    # 0.12345678901234568  ← 末尾已经不对了
```

■ 经典陷阱

python

`0.1 + 0.2` `# 0.30000000000000004` (不是 0.3!)

- 原因：0.1 和 0.2 在二进制里都是无限循环小数，存储时各自有微小误差

```
from decimal import Decimal
```

你以为存的是

0.1

0.2

0.3

实际存的是

→ 0.1000000000000000000555... (比 0.1 大一点点)

→ 0.2000000000000000001110... (比 0.2 大一点点)

→ 0.299999999999999998889... (比 0.3 小一点点)

```
print(Decimal(0.1))    # 0.100000000000000000055511151231...
```

```
print(Decimal(0.2))    # 0.200000000000000000111022302462...
```

```
print(Decimal(0.3))    # 0.29999999999999999888977697537...
```

浮点数类型

■ 如何正确比较浮点数？

python

错误写法

```
0.1 + 0.2 == 0.3 # False
```

正确写法：判断差值是否足够小

计算绝对值 `abs((0.1 + 0.2) - 0.3) < 1e-9` # True

或用 `math.isclose()`

```
import math
```

```
math.isclose(0.1 + 0.2, 0.3) # True
```

⚠ 这不是 Python 的 bug，而是所有编程语言中浮点数的共同特性

■ 定义 & 基本写法

- 复数由**实部**和**虚部**组成，形式为 $a + bi$ ，其中 $i = \sqrt{-1}$
- Python 中用 j 代替数学中的 i

直接用字面量写

```
z1 = 12.3 + 4j
```

```
z2 = -5.6 + 7j
```

```
z3 = 3j
```

实部为 0，即 $0 + 3j$

```
z4 = 5 + 0j
```

虚部为 0，但类型仍然是 `complex`

用 `complex()` 函数创建

```
z5 = complex(12.3, 4)
```

等价于 $12.3 + 4j$

```
z6 = complex(5)
```

等价于 $5 + 0j$

复数类型

■ 获取实部和虚部

```
z = 12.3 + 4j
```

```
z.real      # 12.3      实部 (float 类型)
```

```
z.imag      # 4.0      虚部 (float 类型)
```

注意：实部和虚部都是 float，即使看起来是整数

```
type(z.real) # <class 'float'>
```

```
type(z.imag) # <class 'float'>
```

■ 复数运算

加法和减法

规则：实部与实部运算，虚部与虚部运算。

$$(a + bj) + (c + dj) = (a + c) + (b + d)j$$

$$(a + bj) - (c + dj) = (a - c) + (b - d)j$$

python

```
a = 1 + 2j
```

```
b = 3 + 4j
```

```
a + b      # (4+6j)      实部：1+3=4，虚部：2+4=6
```

```
a - b      # (-2-2j)     实部：1-3=-2，虚部：2-4=-2
```

复数运算

乘法

规则：像展开 $(a + b)(c + d)$ 一样展开，然后把 j^2 替换为 -1 。

$$(a + bj)(c + dj) = ac + adj + bcj + bdj^2 = (ac - bd) + (ad + bc)j$$

以 $(1+2j) \times (3+4j)$ 为例：

展开项	计算	结果
$a \times c$	1×3	3 (实部)
$a \times dj$	$1 \times 4j$	4j (虚部)
$bj \times c$	$2j \times 3$	6j (虚部)
$bj \times dj$	$2j \times 4j = 8j^2 = 8 \times (-1)$	-8 (实部)

实部合并： $3 + (-8) = -5$ ，虚部合并： $4 + 6 = 10$

■ 复数运算

$$a = 1 + 2j$$

$$b = 3 + 4j$$

$$a + b \quad \# (4+6j) \quad \text{加法: 实部加实部, 虚部加虚部}$$

$$a - b \quad \# (-2-2j) \quad \text{减法: 同理}$$

$$a * b \quad \# (-5+10j) \quad \text{乘法: } (1 \times 3 - 2 \times 4) + (1 \times 4 + 2 \times 3)j$$

$$a / b \quad \# (0.44+0.08j) \quad \text{除法}$$

数字类型的操作

■ 内置的数值运算操作符

操作符	描述
$x + y$	x与y之和
$x - y$	x与y之差
$x * y$	x与y之积
x / y	x与y之商，结果为浮点数，4/2结果为2.0，y不能为0
$x // y$	x与y之整数商，即：不大于x与y之商的最大整数，结果为整型
$x \% y$	x与y之商的余数，也称为模运算
$-x$	x的负值，即： $x * (-1)$
$+x$	x本身
$x ** y$	x的y次幂，即： x^y

内置的数值运算操作符

■ 数字类型之间的关系与混合运算

- 整数 -> 浮点数 -> 复数 (从窄到宽)
- 整数是浮点数特例 (小数部分为 0)
- 浮点数是复数特例 (虚部为 0)

3 → 3.0 → (3+0j)
int float complex

内置的数值运算操作符

■ 数字类型之间的关系与混合运算

- 混合运算规则

运算组合	结果类型	示例
int + int	int	$3 + 4 \rightarrow 7$
int × int	int	$3 * 4 \rightarrow 12$
int + float	float	$3 + 1.5 \rightarrow 4.5$
int × float	float	$3 * 1.5 \rightarrow 4.5$
int + complex	complex	$3 + 4j \rightarrow (3+4j)$
float + complex	complex	$1.5 + 4j \rightarrow (1.5+4j)$

⚠ 整数除法 is 唯一的例外，即使两个整数能整除，结果也是 float，例如 $6 / 3 = 2.0$



Python运算符优先级

类别	优先级	运算符	说明	结合方向
分组	1	<code>()</code>	括号	—
算术运算	2	<code>**</code>	幂运算	← 右结合
	3	<code>+x</code> <code>-x</code>	正负号（一元）	—
	4	<code>*</code> <code>/</code> <code>//</code> <code>%</code>	乘、除、整除、取模	→ 左结合
	5	<code>+</code> <code>-</code>	加、减（二元）	→ 左结合
比较运算	6	<code><</code> <code><=</code> <code>></code> <code>>=</code> <code>==</code> <code>!=</code>	比较	→ 左结合
逻辑运算	7	<code>not</code>	逻辑非	—
	8	<code>and</code>	逻辑与	→ 左结合
	9	<code>or</code>	逻辑或	→ 左结合

Python运算符优先级

■ 注意事项

- ****** 是右结合的，其他算术运算符都是左结合， $2^{**}3^{**}4 == 2^{**}(3^{**}4)$
- 如果左侧有正负号，那么幂运算符优先，如果右侧有正负号，那么一元运算符优先， $-3^{**}2 = -9$ 相当于 -3^2 ，而 $3^{**-}2 = 1/9$ ，相当于 3^{-2}

Python运算符优先级

■ 注意事项

- **// 整除：向下取整，而非向零取整**

表达式	数学除法	Python <code>//</code> (向下)	说明
<code>7 // 2</code>	3.5	3	正数时向下 = 向零，结果一致
<code>-7 // 2</code>	-3.5	-4	向下取到 -4，不是 -3
<code>7 // -2</code>	-3.5	-4	同上
<code>-7 // -2</code>	3.5	3	负负得正，结果为正，和正数情况一致

取整方式	规则	-3.5 的结果	谁用
向下取整 (floor)	往数轴 左边 (更小方向) 取	-4	Python
向零取整 (truncate)	往 靠近零 的方向取	-3	C / Java

Python运算符优先级

■ 注意事项

- % 取模：余数符号跟除数一致，// 和 % 始终满足恒等式：

$$a = (a // b) \times b + (a \% b)$$

- ⚠ % 的结果由 // 决定

$$a \% b = a - (a // b) \times b$$

余数 = 被除数 - 商 × 除数

表达式	推导过程	结果	验证
<code>7 % 2</code>	$7 - 3 \times 2$	1	$3 \times 2 + 1 = 7 \checkmark$
<code>-7 % 2</code>	$-7 - (-4) \times 2$	1	$-4 \times 2 + 1 = -7 \checkmark$
<code>7 % -2</code>	$7 - (-4) \times (-2)$	-1	$-4 \times (-2) + (-1) = 7 \checkmark$
<code>-7 % -2</code>	$-7 - 3 \times (-2)$	-1	$3 \times (-2) + (-1) = -7 \checkmark$

Python运算符优先级

■ 注意事项

- **not**、**and**、**or** 优先级各不相同

python

```
True or True and False      # True
```

等价于 True or (True and False), and 先算

```
not True or True            # True
```

等价于 (not True) or True, not 先算

- **⚠** 记不住优先级就加括号。 括号永远是最高优先级，而且代码可读性也更好

■ 练习题

```
result = 2 + 3 ** 2 * 4 - 10 // 3 > 5 and not False
```

Python运算符优先级

■ 练习题

python

```
result = 2 + 3 ** 2 * 4 - 10 // 3 > 5 and not False
```

第1步: ******

$3 ** 2 = 9$

变为: $2 + 9 * 4 - 10 // 3 > 5 \text{ and not False}$

第2步: ***** 和 **//** (从左到右)

$9 * 4 = 36$

$10 // 3 = 3$

变为: $2 + 36 - 3 > 5 \text{ and not False}$

Python运算符优先级

■ 练习题

python

```
result = 2 + 3 ** 2 * 4 - 10 // 3 > 5 and not False
```

第3步: **+** 和 **-** (从左到右)

$2 + 36 = 38$

$38 - 3 = 35$

变为: **$35 > 5$ and not False**

第4步: **>**

$35 > 5 = \text{True}$

变为: **True and not False**

第5步: **not**

$\text{not False} = \text{True}$

变为: **True and True**

第6步: **and**

$\text{True and True} = \text{True}$

增强赋值操作符

■ 定义

- 一种简写形式，将算术运算与赋值合并为一步，使代码更简洁

操作符	描述	示例	结果
<code>+=</code>	加法赋值	<code>x=10; x+=3</code>	<code>x → 13</code>
<code>-=</code>	减法赋值	<code>x=10; x-=3</code>	<code>x → 7</code>
<code>*=</code>	乘法赋值	<code>x=10; x*=3</code>	<code>x → 30</code>
<code>/=</code>	除法赋值	<code>x=10; x/=3</code>	<code>x → 3.333...</code>
<code>//=</code>	取整除赋值	<code>x=10; x//=3</code>	<code>x → 3</code>
<code>%=</code>	取模赋值	<code>x=10; x%=3</code>	<code>x → 1</code>
<code>**=</code>	幂赋值	<code>x=10; x**=2</code>	<code>x → 100</code>

⚠ 二元运算操作符和 `=` 之间没有空格 (`+=` ✓, `+ = x`)

⚠ Python 没有 `++` 和 `--` 运算符，自增请使用 `x += 1`

数字类型的转换

- 数值运算操作符可以隐式地转换输出结果的数字类型
例：两个整数采用运算符 “/” 除法输出浮点数结果
- 通过内置的数字类型转换函数可以显式地在数字类型之间进行转换

函数	描述
int(x)	将x转换为整数，x可以是浮点数或字符串
float(x)	将x转换为浮点数，x可以是整数或字符串

数字类型的转换

■ 整数和浮点数之间可以相互转换

函数：int(), float()

示例：

- `int(4.5) = 4` （直接去掉小数部分，不使用四舍五入）
- `int(-4.5) = -4` (负数也是直接去掉小数部分)
- `float(4) = 4.0` （增加小数部分）

■ 字符串也可以转换为整数和浮点数

`int('123.123')` 不合法

`int(float('1213.13'))` 合法

`int()`只能转换纯整数数字的字符串

`float()`能转换纯整数/浮点数字符串

数字类型的转换

■ round 函数

- 用于将一个数值 “四舍五入” 到指定的小数位数
- 不是传统的 “四舍五入”，而是 “四舍六入五取偶”。当要舍弃的部分恰好是 5 时，它会取最近的偶数
- 基本语法：round(number, ndigits)
- number 是要处理的数值，ndigits 是保留的小数位数 (可选，默认为 0，此时返回整数)

round(3.1415, 2)	# → 3.14	保留2位小数
round(3.1415, 1)	# → 3.1	保留1位小数
round(3.1415)	# → 3	不传第二个参数，返回整数
round(1234, -2)	# → 1200	ndigits为负数，对整数部分取近似

数字类型的转换

■ round 函数

- 不是传统的“四舍五入”，而是“**四舍六入五取偶**”。当要舍弃的部分恰好是 5 时，它会取**最近的偶数**

保留到个位，舍弃部分恰好是 .5

`round(0.5)` # 保留位是0（偶），舍去 → 0

`round(1.5)` # 保留位是1（奇），进位 → 2

`round(2.5)` # 保留位是2（偶），舍去 → 2

`round(3.5)` # 保留位是3（奇），进位 → 4

数字类型的转换

■ round 函数

- 不是传统的“四舍五入”，而是“四舍六入五取偶”。当要舍弃的部分恰好是 5 时，它会取最近的偶数

保留到十位，舍弃部分恰好是 5（个位）

`round(1235, -1)` # 保留位是3（奇），进位 → 1240

`round(1245, -1)` # 保留位是4（偶），舍去 → 1240

保留1位小数，舍弃部分恰好是 .05

`round(2.25, 1)` # 保留位是2（偶），舍去 → 2.2

`round(2.35, 1)` # 保留位是3（奇），进位 → 2.4

- 设计原因：
 - 传统四舍五入遇到 5 永远进位，大量累加后结果会系统性偏大
 - 统计上误差更均匀

数字类型的转换

■ 四舍五入

- **decimal** 标准库：提供精确的十进制浮点运算，专门用来解决 float 不精确的问题
- **ROUND_HALF_UP**：传统四舍五入策略常量

```
from decimal import Decimal, ROUND_HALF_UP
```

保留到个位

```
Decimal('2.5').quantize(Decimal('1'), rounding=ROUND_HALF_UP)      # → 3
Decimal('3.5').quantize(Decimal('1'), rounding=ROUND_HALF_UP)      # → 4
Decimal('-2.5').quantize(Decimal('1'), rounding=ROUND_HALF_UP)     # → -3
```

保留1位小数

```
Decimal('2.25').quantize(Decimal('0.1'), rounding=ROUND_HALF_UP)   # → 2.3
Decimal('2.35').quantize(Decimal('0.1'), rounding=ROUND_HALF_UP)   # → 2.4
```

- **quantize()** 的参数控制精度：Decimal('1') 是个位，Decimal('0.01') 是两位小数，Decimal('100') 是百位，以此类推
- 必须用**字符串构造 Decimal**，若直接传入浮点数类型，舍入结果会出错

数字类型的转换

■ 四舍五入

```
from decimal import Decimal, ROUND_HALF_UP
```

```
# 先看看 Decimal(1.15) 到底存了什么
```

```
print(Decimal(1.15))
```

```
# → 1.149999999999999911182158029987476766109466552734375
```

```
# ✅ 字符串构造 - 正确
```

```
print(Decimal('1.15').quantize(Decimal('0.1'), rounding=ROUND_HALF_UP))
```

```
# → 1.2 ✓
```

```
# ❌ 浮点数构造 - 出错
```

```
print(Decimal(1.15).quantize(Decimal('0.1'), rounding=ROUND_HALF_UP))
```

```
# → 1.1 ✗
```

数字类型的判断

- 函数：**type(x)**，返回x的类型，适用于所有类型的判断

```
>>> a = 1                >>> c = 3.14
>>> type(a)              >>> type(c)
<class 'int'>            <class 'float'>

>>> b = "hello"          >>> d = True
>>> type(b)              >>> type(d)
<class 'str'>            <class 'bool'>
```

- 函数：**isinstance(x, type)**，判断 x 是不是某种类型

```
>>> b = "hello"
>>> isinstance(b, str)
True
>>> isinstance(b, int)
False
>>> isinstance(b, float)
False
```



数字类型的判断

■ Python有多重内置数据类型

- **数字类型** : int, float, complex
- **布尔类型** : bool (取值True或False)
- **字符串类型** : str
- **组合数据类型** : list, tuple, set, dict
- **二进制数据类型** : bytes(表示字节序列) , bytearray(可变的字节序列) , memoryview(内存视图类型)
- **None类型** : NoneType(表示无或缺失值 , 唯一的值为None)

■ 自定义数据类型 , 通过创建**类**来实现



内置的数值运算函数

Python解释器提供了一些内置函数，在这些内置函数之中，有6个函数与数值运算相关

函数	描述
<code>abs(x)</code>	x的绝对值
<code>divmod(x, y)</code>	$(x//y, x\%y)$ ，输出为二元组形式（也称为元组类型）
<code>pow(x, y[, z])</code>	$(x**y)\%z$ ，[.]表示该参数可以省略，即： <code>pow(x,y)</code> ，它与 $x**y$ 相同
<code>round(x[, ndigits])</code>	对x四舍五入，保留ndigits位小数。 <code>round(x)</code> 返回四舍五入的整数
<code>max(x₁, x₂, ..., x_n)</code>	x ₁ , x ₂ , ..., x _n 的最大值，n没有限定
<code>min(x₁, x₂, ..., x_n)</code>	x ₁ , x ₂ , ..., x _n 的最小值，n没有限定

内置的数值运算函数

```
>>> abs(-5)
5                                # 返回绝对值, 去掉负号
>>> abs(3.14)
3.14                            # 正数的绝对值是它本身

>>> divmod(13, 3)
(4, 1)                          # 13 ÷ 3 = 商4余1, 等价于 (13//3, 13%3)
>>> divmod(-7, 2)
(-4, 1)                        # 商向负无穷取整, 和 // 行为一致

>>> max(3, 7, 2, 9, 1)
9                                # 返回多个参数中的最大值
>>> max([10, 20, 30])
30                              # 也可以传入列表

>>> min(3, 7, 2, 9, 1)
1                                # 返回多个参数中的最小值
>>> min([10, 20, 30])
10                              # 也可以传入列表
```



Python内置函数、标准库、第三方库

■ 内置函数

- Python解释器给用户提供一些常用函数，可直接使用 (print, int, max 等)
- `dir()` 与 `__builtins__` :
 - `dir()` 函数：列出一个对象拥有的所有名字（属性、方法、子对象等），返回按字母排序的列表
 - `__builtins__`：Python 解释器启动时自动加载的一个内置模块，包含所有不需要 import 就能直接使用的名字
 - `dir(__builtins__)`：列出这个模块里的全部名字



Python内置函数、标准库、第三方库

■ 内置函数

➤ 从`dir(__builtins__)`中筛选出内置函数：

第一步：查看全部名字

```
>>> len(dir(__builtins__))
```

```
157
```

共157个，不全是函数

第二步：观察规律，按首字母分三类

下划线开头 → 8个，特殊属性（如 `__doc__`, `__name__`）

大写字母开头 → 74个，异常类和常量（如 `ValueError`, `True`, `None`）

小写字母开头 → 75个，内置函数（如 `print`, `len`, `int`）

第三步：用列表推导式筛选出内置函数

```
>>> funcs = [name for name in dir(__builtins__) if name[0].islower()]
```

```
>>> len(funcs)
```

```
75
```



Python内置函数、标准库、第三方库

■ 标准库(内置函数库)

- 相当于Python解释器的外部扩展，随着python安装时**默认自带**的库，但不会随着解释器的启动而启动
- 用之前须提前导入 **import <库名>**
- 提供了操作系统功能、网络通信、文本处理、文件处理、数学运算等基本功能

■ 第三方库

- **需要下载**安装的库，使用方式和标准库类似，需要用import语句调用
- 覆盖了我们能想象到的所有领域，如科学计算、WEB开发、大数据、人工智能、图形系统等



Python内置函数、标准库、第三方库

	内置函数	标准库	第三方库
使用方式	直接调用	<code>import</code> 后使用	<code>pip install</code> + <code>import</code>
示例	<code>print()</code> , <code>len()</code>	<code>math</code> , <code>os</code> , <code>json</code>	<code>numpy</code> , <code>pandas</code> , <code>requests</code>
数量	75个	~300个模块	~90万个包 (PyPI)
维护者	Python官方	Python官方	社区开发者



练习题

已知 $x = -17$, $y = 4$, 写出以下表达式的值, 并说明每一步的计算过程。

python

```
result = round(x / y) + x // y + int(x / y) + abs(x) % y
```

练习题

已知 $x = -17$, $y = 4$, 写出以下表达式的值, 并说明每一步的计算过程。

python

```
result = round(x / y) + x // y + int(x / y) + abs(x) % y
```

第一步: 算公共部分

$x / y = -17 / 4 = -4.25$, $/$ 的结果始终为 float。

第二步: $\text{round}(-4.25)$

保留到个位, 舍弃部分是 0.25。0.25 < 0.5, 不到一半, 舍去 $\rightarrow -4$

第三步: $x // y = -17 // 4$

$//$ 是向下取整 (floor), -4.25 向负无穷方向取整 $\rightarrow -5$

练习题

已知 $x = -17$, $y = 4$, 写出以下表达式的值, 并说明每一步的计算过程。

python

```
result = round(x / y) + x // y + int(x / y) + abs(x) % y
```

第四步: $\text{int}(x / y) = \text{int}(-4.25)$

$\text{int}()$ 向零截断, 直接砍掉小数部分 $\rightarrow -4$

第五步: $\text{abs}(x) \% y = \text{abs}(-17) \% 4 = 17 \% 4$

余数 = 被除数 - 商 $\times$ 除数 $= 17 - 4 \times 4 = 1$

第六步: 合并

$\text{result} = (-4) + (-5) + (-4) + 1 = -12$

math库

- math库是Python提供的内置数学类函数库
- math库不支持复数类型，仅支持实数运算 (整数和浮点数)
- 复数运算需使用 cmath 库
- math库提供了5 个数学常数和 56 个函数 (Python 3.12)

math库

- math库提供了5 个数学常数和 56 个函数 (Python 3.12)

```
import math
```

```
names = [n for n in dir(math) if not n.startswith('_')]
constants = [n for n in names if not callable(getattr(math, n))]
functions = [n for n in names if callable(getattr(math, n))]

print(f'常数: {len(constants)}个')
print(f'函数: {len(functions)}个')
```

■ 5 个数学常数

常数	值	含义
<code>math.pi</code>	3.14159...	圆周率 π
<code>math.e</code>	2.71828...	自然常数 e
<code>math.tau</code>	6.28318...	$\tau = 2\pi$
<code>math.inf</code>	∞	正无穷大
<code>math.nan</code>	NaN	非数值

■ 56 个函数

分类	数量	作用
数论与整数	9	取整、阶乘、最大公约数、排列组合等整数相关运算
幂与对数	10	幂、平方根、指数、各种底数的对数
三角函数	7	正弦、余弦、正切及其反函数
双曲函数	6	双曲正弦、余弦、正切及其反函数
角度转换	2	弧度与角度的互相转换
浮点数工具	10	浮点取模、拆分、精确求和、符号操作等底层浮点处理
浮点数判断	4	判断是否有限、无穷、NaN，以及两数是否足够接近
其他	8	距离计算、连乘、伽马函数、误差函数等

■ 56 个函数

- 数论与整数

<code>math.ceil(2.3)</code>	<code># → 3</code>	向上取整
<code>math.floor(2.7)</code>	<code># → 2</code>	向下取整
<code>math.factorial(5)</code>	<code># → 120</code>	阶乘 5!
<code>math.gcd(12, 8)</code>	<code># → 4</code>	最大公约数
<code>math.comb(5, 2)</code>	<code># → 10</code>	组合数 $C(5, 2)$

- 幂与对数

<code>math.sqrt(16)</code>	<code># → 4.0</code>	平方根
<code>math.pow(2, 3)</code>	<code># → 8.0</code>	幂运算, 返回浮点数
<code>math.exp(1)</code>	<code># → 2.718</code>	e的x次方
<code>math.log(100, 10)</code>	<code># → 2.0</code>	对数, 第二个参数是底数
<code>math.log2(8)</code>	<code># → 3.0</code>	以2为底的对数

random库概述

- 随机数应用普遍，如游戏随机事件、验证码生成等
- python的标准库之一，使用前需 `import random`
- 用于产生各种分布的伪随机数序列（由算法生成，并非真正随机）
- 采用梅森旋转算法（Mersenne twister）生成，可满足除加解密外的大多数工程应用
- 提供了不同类型的随机数函数，所有函数都是基于最基本的 `random.random()` 函数扩展而来

random库解析

分类	函数	描述	示例
种子控制	<code>random.seed(n)</code>	设置随机种子，相同种子产生相同的随机序列	<code>random.seed(42)</code>
数值生成	<code>random.randint(a, b)</code>	生成 [a, b] 之间的随机整数	<code>random.randint(1, 10) → 7</code>
	<code>random.random()</code>	生成 [0.0, 1.0) 之间的随机浮点数	<code>random.random() → 0.5732</code>
	<code>random.uniform(a, b)</code>	生成 a 到 b 之间的随机浮点数	<code>random.uniform(1.0, 5.0) → 3.217</code>
序列操作	<code>random.choice(seq)</code>	从序列中随机取一个元素	<code>random.choice(["a","b","c"]) → "b"</code>
	<code>random.choices(seq, k=n)</code>	随机取 k 个元素，可重复，支持权重	<code>random.choices(["a","b"], k=3) → ["a","a","b"]</code>
	<code>random.sample(pop, k)</code>	随机取 k 个元素，不重复	<code>random.sample(range(100), 5) → [23, 7, 91, 45, 62]</code>
	<code>random.shuffle(seq)</code>	将序列原地随机打乱，无返回值	<code>random.shuffle(ls)</code>

random库解析

- 生成随机数之前可以通过seed()函数指定随机数种子
- 随机种子一般是一个整数，只要种子相同，每次生成的随机数序列也相同

```
import random
```

第一组：种子设为 10

```
random.seed(10)
print(random.randint(1, 100)) # 74
print(random.randint(1, 100)) # 5
print(random.randint(1, 100)) # 55
```

第二组：相同种子，相同调用 → 相同结果

```
random.seed(10)
print(random.randint(1, 100)) # 74
print(random.randint(1, 100)) # 5
print(random.randint(1, 100)) # 55
```

第三组：不同种子 → 不同结果

```
random.seed(20)
print(random.randint(1, 100)) # 93
print(random.randint(1, 100)) # 1
print(random.randint(1, 100)) # 80
```

第四组：相同种子，但调用不同函数 → 结果也不同

```
random.seed(10)
print(random.random()) # 0.5714025946899135
print(random.randint(1, 50)) # 27
print(random.uniform(1, 10)) # 6.82
```

random库解析

■ sample 与 choices 函数

- random.sample(seq, k)
 - 从序列中随机取 k 个元素，每个元素最多被选一次

```
# sample: 不重复抽取 3 个  
prizes = ["金币", "银币", "铜币", "钻石", "宝石"]  
print(random.sample(prizes, 3))    # ['钻石', '金币', '宝石'] (不会重复)
```

⚠ k 不能超过序列长度，否则报错

random库解析

■ sample 与 choices 函数

- `random.choices(seq, weights, k=n)`
 - 从序列中随机取 `k` 个元素，同一元素可以被多次选中，还可以设置每个元素被抽中的概率

```
import random
```

```
# choices: 可重复抽取 3 个（等概率）
```

```
prizes = ["金币", "银币", "铜币", "钻石", "宝石"]
```

```
print(random.choices(prizes, k=3)) # ['金币', '金币', '钻石']（可能重复）
```

```
# choices: 设置权重，钻石和宝石更难抽到
```

```
weights = [30, 30, 25, 10, 5]
```

```
print(random.choices(prizes, weights=weights, k=3)) # ['金币', '铜币', '银币']
```

■ 霍格沃茨魔法学院迎新日

霍格沃茨魔法学院迎来新生入学日，请用 Python 模拟以下场景：

1. 为保证每次运行结果一致，设置随机种子为 2026
2. 新生到达后，分院帽从四个学院（格兰芬多、赫奇帕奇、拉文克劳、斯莱特林）中随机分配一个学院
3. 魔药课上配制魔药，随机生成一个 0~1 之间的浮点数作为魔药浓度（保留两位小数）
4. 新生参加魔药课入学测试，随机生成一个 60~100 之间的整数成绩
5. 新生领取飞天扫帚，随机生成一个 50.0~150.0 之间的浮点数作为扫帚速度（保留一位小数）
6. 魁地奇比赛，将5名选手（哈利、罗恩、赫敏、马尔福、卢娜）的出场顺序随机打乱
7. 从5件奖品（隐形斗篷、活点地图、魔杖、凤凰羽毛、时间转换器）中不重复抽取2件作为入学礼物
8. 魔法商店开业抽奖，从上述5件奖品中可重复抽取3件，其中各奖品的抽中权重分别为 5、10、30、35、20

■ 霍格沃茨魔法学院迎新日

输出示例：

分院帽说：你属于【拉文克劳】！

魔药浓度：0.36

魔药课成绩：87 分

飞天扫帚速度：123.4 km/h

魁地奇出场顺序：['卢娜', '哈利', '马尔福', '赫敏', '罗恩']

奖品抽取（不重复）：['活点地图', '时间转换器']

魔法商店抽奖（可重复）：['凤凰羽毛', '魔杖', '凤凰羽毛']

random库解析

■ 霍格沃茨魔法学院迎新日

```
import random
```

```
# seed(n): 设置随机种子, 固定结果, 方便课堂演示
```

```
random.seed(2026)
```

```
# choice(seq): 从序列中随机取一个元素
```

```
houses = ["格兰芬多", "赫奇帕奇", "拉文克劳", "斯莱特林"]
```

```
my_house = random.choice(houses)
```

```
print(f"分院帽说: 你属于 [{my_house}] ! ")
```

```
# random(): 生成 [0.0, 1.0) 之间的随机浮点数
```

```
potion = random.random()
```

```
print(f"魔药浓度: {potion:.2f}")
```

```
# randint(a, b): 生成 [a, b] 之间的随机整数
```

```
score = random.randint(60, 100)
```

```
print(f"魔药课成绩: {score} 分")
```

```
# uniform(a, b): 生成 a 到 b 之间的随机浮点数
```

```
speed = round(random.uniform(50.0, 150.0), 1)
```

```
print(f"飞天扫帚速度: {speed} km/h")
```

■ 霍格沃茨魔法学院迎新日

shuffle(seq): 将序列原地随机打乱

```
players = ["哈利", "罗恩", "赫敏", "马尔福", "卢娜"]
```

```
random.shuffle(players)
```

```
print(f"魁地奇出场顺序: {players}")
```

sample(seq, k): 不重复抽取 k 个元素

```
prizes = ["隐形斗篷", "活点地图", "魔杖", "凤凰羽毛", "时间转换器"]
```

```
my_prizes = random.sample(prizes, 2)
```

```
print(f"奖品抽取 (不重复): {my_prizes}")
```

choices(seq, weights, k=n): 可重复抽取 k 个元素, 支持权重

```
weights = [5, 10, 30, 35, 20]
```

```
result = random.choices(prizes, weights=weights, k=3)
```

```
print(f"魔法商店抽奖 (可重复): {result}")
```